B23-L6 lab: Train a PyTorch model on survey data

Platform: Laptop

Work in `labs/torch/` in this repository. It holds a seeded generator of survey data, a training script and a logistic regression baseline. You write the steps from module B23 and from lessons B16-L2 and B16-L3: preprocessing, the dataset, the model, one training epoch, evaluation and early stopping. You need PyTorch 2.15 for the laptop: it is not in the app or the `dev` group. When the model beats the baseline, push the lab to your own GitHub repository with a README: it is the roadmap's PyTorch evidence for month 3.

1. Run `cd labs/torch`, create a virtual environment, then `pip install -e .` and `pip install pytest`. If the PyTorch wheel is large for your machine, install the CPU build first: `pip install torch --index-url https://download.pytorch.org/whl/cpu`. Run `pytest`: most tests fail with NotImplementedError. Read `README.md` and every file in `src/survey_torch/`.
2. In `src/survey_torch/prep.py`, write `split_indices` and `Standardiser` (lessons B23-L6 and B10-L2). Run `pytest tests/test_prep.py -k "split or standard"`.
3. Write `EarlyStopping` in the same file (lesson B16-L2). It must keep a deep copy of the best state.
4. In `src/survey_torch/nn_steps.py`, write `SurveyRows` and `make_model` (lessons B23-L1, B23-L4 and B23-L6), then `train_one_epoch` and `evaluate` (lessons B23-L3, B23-L5 and B16-L2). Run `pytest`: every test passes.
5. Run `python -m survey_torch train`. It prints the baseline's and your model's validation loss and accuracy. Change `--hidden`, `--lr` and `--patience`, and keep notes on what each change did.
6. Run the same command twice with the same `--seed` and check the numbers match (lesson B16-L3).

7. Write your README: the data, the model, the baseline comparison, a loss curve, and one thing that did not work. Push the folder to a new GitHub repository.

Notes